Security & Performance Review (Days 81-83)

Security review checklist (Day 81)

Go through this list against the actual code — most items are quick to check, a couple need a real fix.

- [ ] **CORS** — not configured anywhere yet. If a frontend will ever call this API from a browser on a different origin, add a `CorsConfigurationSource` bean; skip if this stays API-only/Postman-tested.
- [ ] **JWT expiry** — `jwt.expiration=86400000` (24h). Reasonable for a project; a real bank would be much shorter + refresh tokens. Worth a one-line note in your README about this tradeoff for interviews.
- [ ] **Password policy** — `RegisterRequest` requires 8+ chars, nothing else (no complexity rule). Fine for a demo; mention it's simplified.
- [ ] **Secrets in `application.properties`** — already correctly pulled from env vars (`${DB_PASSWORD}` , `${JWT_SECRET}`) rather than hardcoded. Double-check no `.env` file or real credentials are committed to git: `git log --all -- .env` should return nothing.
- [ ] **Audit log leakage** — `AuditLog.oldValue` / `newValue` store things like `"email=..."` — never passwords (confirmed: `UserServiceImpl` never logs the password field). Good as-is.
- [ ] **Error messages** — `GlobalExceptionHandler` 's catch-all already returns a generic 500 message instead of leaking stack traces. Good as-is.
- [ ] **SQL injection** — all repository access goes through Spring Data JPA (parameterized queries), no raw string-concatenated SQL anywhere in the codebase. Verified clean by construction.
- [ ] **Rate limiting** — none exists on `/auth/login` (brute-force risk). Out of scope to build now, but worth naming as a known gap.

Optimistic locking (Day 82)

`Account.version` (`@Version`) has existed since Day 16 but has never actually been tested under real concurrency. Prove it works:

```
@Test
void concurrentUpdates_oneWinsOneThrowsOptimisticLock() throws Exception {
    Account account = accountRepository.save(Account.builder()
            .balance(BigDecimal.valueOf(1000))...build());

    Account copy1 = accountRepository.findById(account.getId()).orElseThrow();
    Account copy2 = accountRepository.findById(account.getId()).orElseThrow();

    copy1.setBalance(copy1.getBalance().add(BigDecimal.TEN));
    accountRepository.saveAndFlush(copy1);   // succeeds, version increments

    copy2.setBalance(copy2.getBalance().add(BigDecimal.TEN));
    assertThrows(ObjectOptimisticLockingFailureException.class,
            () -> accountRepository.saveAndFlush(copy2));   // stale version — rejected
}
```

Manual test (no code, just curl): open two terminals, both do `GET /api/v1/accounts/1` to load the same account, then fire two `POST /api/v1/transactions/deposit` requests for the same account nearly simultaneously (`curl ... & curl ... & wait`). One should succeed; if the second also silently succeeds instead of retrying or erroring, optimistic locking isn't actually engaged on that write path — worth checking `AccountServiceImpl`'s deposit/withdraw save calls go through the versioned entity, not a detached stale copy.

Performance review (Day 83)

- **N+1 queries**: with `spring.jpa.show-sql=true`, hit `GET /api/v1/transactions/account/{id}/history` with several transactions on the account — count the SQL statements logged. If you see one query per transaction (to lazily load `sourceAccount` / `destinationAccount`) instead of one query total, that's N+1. Fix with `@EntityGraph` or a `JOIN FETCH` in the repository query if it shows up.
- **Connection pool**: default HikariCP pool size is 10 — fine for a demo, just know it's there (`spring.datasource.hikari.maximum-pool-size` to tune later).
- **Indexes**: covered in doc 04 (Flyway/indexes).